Umm Al-Qura University
College of Computing
Computer Science and AI Department

Data management and representation
second Semester 2024

# Data management and Representation project

Date: 30/1/2025

Group: 1

TEAM MEMBERS' NAMES:

Mawaddah Alshebly
445013878

Refan Almfarriji
445000906

Batoul Hallaq
445013931

Rawaf Tazi
445003909

COLAB NOTEBOOK:

https://colab.research.google.com/drive/1bGbNCXDQTGEsMf4vBHPEGLzd5C6_wpHO?usp=sharing

# Table of Contents

1.Introduction .....3
2. Data exploration .....4
3. Libraries .....5
4.Data cleaning .....6
5.Data analysis.....11
6.Visualisation .....13
7.Modelling .....20
8. Conclusion.....24

# 1.INTRODUCTION

## Problem Overview:

The problem is to analyze and model crime data to understand patterns, trends, and relationships between different factors related to crimes. This understanding can be used to predict future crime occurrences or to help law enforcement agencies make more informed decisions**.**

## DATASET Overview:

The data represents statistics for a number of crimes that occurred in America between 2020 and 2024. The statistics include the place (area) of the crime, its date and time, as well as the type of crime and the weapon used in it. In addition, the statistics include the ages and genders of the victims. The original data was imported from the Kaggle website, which included **1,000,000 rows** and **28 columns** in the table, and then we took random samples of **30,000** rows.

## OBJECTIVES:

- Analyze crime patterns: Understand the relationship between the type of crime and influencing factors such as the type of weapon, time, date, and area.
- Target age group analysis: Understanding which age groups are most vulnerable to certain types of crimes.
- Build a model (Random Forest) to predict the type of crime and victim's age based on other factors

# 2. DATA EXPLORATION

## 1. CRIME DISTRIBUTION AND FREQUENCY:

- **Area:** Harbor and Mission areas have the highest crime counts, suggesting they might be crime hotspots.
- **Type of crime:** VIOLENCE is the most frequent type of crime, followed by TERRORIZATION THEFT.
- **Seriousness:** The majority of crimes are not classified as serious ('No').
- **Gender:** Females are more frequently victims of crimes compared to males.
- **Time:** Most crimes happen between 6PM - 9PM.
- **Age:** The 30-39 age group is most frequently victimized.
- **Weapon:** The most common weapon used is ' PHYSICAL VIOLENCE '.

## 2. RELATIONSHIPS BETWEEN VARIABLES:

- **Area and Crime Type:** Certain crime types are more prevalent in specific areas (e.g., THEFT and VIOLENCE is highest in Harbor).
- **Time and Seriousness:** Serious crimes are more likely to occur between 6 PM and 11 AM, this timeframe requires special attention for crime prevention.
- **Victim Age:** Violence and terrorization crimes occur at all ages, while property damages and behavioral crimes are less frequent in older victims
- **Crime Type and Weapon:** Certain weapons are commonly associated with specific crime types (e.g., PHYSICAL VIOLENCE with VIOLENCE crimes).

## 3. OTHER FINDINGS:

- **Time Series:** There has been a general trend of decreasing crime rates over the years 2022-2024.
- **Victim Age Statistics:** The mean age of the victims is 37, while the median age is 36, with a standard deviation of approximately 15.

## OVERALL, THE EDA REVEALS IMPORTANT PATTERNS AND TRENDS IN THE CRIME DATA:

- VIOLENCE is the most common crime type, with HARBOR and MISSION being areas of high crime occurrence.
- There's a time dependency in crime seriousness and type.
- Females are more likely to be victims, and younger individuals, particularly those in the 30-39 age group, are frequently targeted.
- Crime rates seem to be decreasing over recent years.

These insights can guide further investigation and decision-making for crime prevention and law enforcement strategies.

# 3. LIBRARIES

```
# Importing necessary libraries
import pandas as pd
import numpy as np
from datascience import *
from datetime import datetime
%matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('fivethirtyeight')
!pip install seaborn #Use 'pip' tool to install the seaborn library
import seaborn as sns
!pip install --upgrade scikit-learn #Use 'pip' tool to install the scikit-learn library
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler , MinMaxScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.multioutput import MultiOutputClassifier
```

- **Pandas**: which is used to manipulate data and tables.
- **NumPy:** which is used to work with arrays and mathematical operations.
- **Datascience:** which is used to analyze data and tables.
- **Datetime:** to handle dates and times.
- **%matplotlib online:** a command used in Jupyter Notebook to display graphs directly within the notebook.
- **Matplotlib:** for creating graphs and visualizations.
- **plt.style.use('fivethirtyeight'):** Choose an infographic design style inspired by FiveThirtyEight.
- **Seaborn:** for use in graphic visualizations.
- **train_test_split:** to split the data into training and test.
- **sklearn.preprocessing:** LabelEncoder, StandardScaler, MinMaxScaler:
  - **LabelEncoder:** Converts text values to numbers.
  - **StandardScaler:** Standardizes the data so that the mean is 0 and the standard deviation is 1.
  - **MinMaxScaler:** Normalizes data within a specific range (usually between 0 and 1).
- **RandomForestClassifier:** a machine learning model that uses random forests for classification.
- **MultiOutputClassifier:** which is used when predicting more than one output at the same time.

# 4.DATA CLEANING

```
[ ] # Removing unnecessary columns
    data = data.drop('DR_NO','Date Rptd', 'AREA', 'Rpt Dist No', 'Crm Cd', 'Mocodes', 'Vict Descent', 'Premis Cd',
                     'Weapon Used Cd', 'Status', 'Crm Cd 1', 'Crm Cd 2', 'Crm Cd 3', 'Crm Cd 4', 'LOCATION', 'Cross Street',
                     'LAT', 'LON','Premis Desc', 'Status Desc')
    data.show(5)
```

| DATE OCC | TIME OCC | AREA NAME | Part 1-2 | Crm Cd Desc | Vict Age | Vict Sex | Weapon Desc |
|---|---|---|---|---|---|---|---|
| 3/1/2020 0:00 | 2130 | Wilshire | 1 | VEHICLE - STOLEN | 0 | M | nan |
| 2/8/2020 0:00 | 1800 | Central | 1 | BURGLARY FROM VEHICLE | 47 | M | nan |
| 11/4/2020 0:00 | 1700 | Southwest | 1 | BIKE - STOLEN | 19 | X | nan |
| 3/10/2020 0:00 | 2037 | Van Nuys | 1 | SHOPLIFTING-GRAND THEFT ($950.01 & OVER) | 19 | M | nan |
| 9/9/2020 0:00 | 630 | Hollenbeck | 1 | VEHICLE - STOLEN | 0 | nan | nan |

... (30240 rows omitted)

```
[ ] # Removing duplicate rows
    data_df = data.to_df()
    data_df = data_df.drop_duplicates()
    data = Table.from_df(data_df)
    data.show(5)
```

| DATE OCC | TIME OCC | AREA NAME | Part 1-2 | Crm Cd Desc | Vict Age | Vict Sex | Weapon Desc |
|---|---|---|---|---|---|---|---|
| 3/1/2020 0:00 | 2130 | Wilshire | 1 | VEHICLE - STOLEN | 0 | M | nan |
| 2/8/2020 0:00 | 1800 | Central | 1 | BURGLARY FROM VEHICLE | 47 | M | nan |
| 11/4/2020 0:00 | 1700 | Southwest | 1 | BIKE - STOLEN | 19 | X | nan |
| 3/10/2020 0:00 | 2037 | Van Nuys | 1 | SHOPLIFTING-GRAND THEFT ($950.01 & OVER) | 19 | M | nan |
| 9/9/2020 0:00 | 630 | Hollenbeck | 1 | VEHICLE - STOLEN | 0 | nan | nan |

... (29916 rows omitted)

In this section, we started cleaning the data. The first code deletes the columns that are not necessary for our data. While the second code deletes duplicate rows from the table. Both codes display the first five rows of the table after execution.

```
[ ] #  Renaming columns
    data = data.relabeled('DATE OCC', 'Date')
    data = data.relabeled('TIME OCC', 'Time')
    data = data.relabeled('AREA NAME', 'Area')
    data = data.relabeled('Part 1-2', 'Serious')
    data = data.relabeled('Crm Cd Desc', 'Type of crime')
    data = data.relabeled('Weapon Desc', 'Weapon')
    data.show(5)
```

| Date | Time | Area | Serious | Type of crime | Vict Age | Vict Sex | Weapon |
|---|---|---|---|---|---|---|---|
| 3/1/2020 0:00 | 2130 | Wilshire | 1 | VEHICLE - STOLEN | 0 | M | nan |
| 2/8/2020 0:00 | 1800 | Central | 1 | BURGLARY FROM VEHICLE | 47 | M | nan |
| 11/4/2020 0:00 | 1700 | Southwest | 1 | BIKE - STOLEN | 19 | X | nan |
| 3/10/2020 0:00 | 2037 | Van Nuys | 1 | SHOPLIFTING-GRAND THEFT ($950.01 & OVER) | 19 | M | nan |
| 9/9/2020 0:00 | 630 | Hollenbeck | 1 | VEHICLE - STOLEN | 0 | nan | nan |

... (29916 rows omitted)

In this section, we changed the names of the remaining columns and renamed them with more clear names.

```
[ ] # Dealting missing values
    def filter_all_columns(data):
        invalid_values = ["nan", '', 0, np.nan,'UNKNOWN WEAPON/OTHER WEAPON']
        for column in data.labels:
          if column in ['Date', 'Time', 'Area', 'Type of crime', 'Status','Vict Age', 'Vict Sex','Weapon']:
            data = data.where(column, lambda value: value not in invalid_values)
        return data

    data = filter_all_columns(data)
    data.show(5)
```

| Date | Time | Area | Serious | Type of crime | Vict Age | Vict Sex | Weapon |
|---|---|---|---|---|---|---|---|
| 5/10/2020 0:00 | 2200 | Central | 2 | INTIMATE PARTNER - SIMPLE ASSAULT | 30 | F | STRONG-ARM (HANDS, FIST, FEET OR BODILY FORCE) |
| 12/2/2020 0:00 | 2245 | 77th Street | 1 | ROBBERY | 21 | M | VERBAL THREAT |
| 5/1/2020 0:00 | 2330 | 77th Street | 1 | ASSAULT WITH DEADLY WEAPON, AGGRAVATED ASSAULT | 28 | F | KNIFE WITH BLADE OVER 6 INCHES IN LENGTH |
| 1/18/2020 0:00 | 940 | Central | 2 | BATTERY - SIMPLE ASSAULT | 42 | M | STRONG-ARM (HANDS, FIST, FEET OR BODILY FORCE) |
| 8/27/2020 0:00 | 1255 | Van Nuys | 2 | BATTERY - SIMPLE ASSAULT | 57 | F | STRONG-ARM (HANDS, FIST, FEET OR BODILY FORCE) |

... (9872 rows omitted)

In this section, we delete rows that contain missing values, zeros, or other unknown or unwanted values.

```
[ ] # Find all unique values within a specific column
    def unique_values(colm):
      for column in data.labels:
        unique_values = np.unique(data.column(colm))
        return unique_values

    # Replace values within a column by provided mapping
    def rename_values(column, mapping):
        return [mapping.get(value, value) for value in column]
```

```
[ ] # Display all distinct values present in 'Area' column
    unique_values('Area')
```

```
array(['77th Street', 'Central', 'Devonshire', 'Foothill', 'Harbor',
       'Hollenbeck', 'Hollywood', 'Mission', 'N Hollywood', 'Newton',
       'Northeast', 'Olympic', 'Pacific', 'Rampart', 'Southeast',
       'Southwest', 'Topanga', 'Van Nuys', 'West LA', 'West Valley',
       'Wilshire'],
      dtype='<U11')
```

```
[ ] # Combining variations into more common forms
    data = data.with_column('Area', data.apply(lambda x: 'Hollywood' if x == 'N Hollywood' else ('West LA' if x == 'West Valley' else x), 'Area'))
    unique_values('Area')
```

```
array(['77th Street', 'Central', 'Devonshire', 'Foothill', 'Harbor',
       'Hollenbeck', 'Hollywood', 'Mission', 'Newton', 'Northeast',
       'Olympic', 'Pacific', 'Rampart', 'Southeast', 'Southwest',
       'Topanga', 'Van Nuys', 'West LA', 'Wilshire'],
      dtype='<U11')
```

A method has been created that extracts distinct values according to the column name that is provided to it. This method was then applied to the column (area) and the results were displayed. Finally, similar values were combined under unified names to become fewer in number and clearer.

```
[ ] # Display all distinct values present in 'Serious' column
    unique_values('Serious')
```

```
array([1, 2])
```

```
[ ] # Converting numerical representation in 'Serious' column into more readable categorical representation ('Yes' or 'No').
    data = data.with_column('Serious', data.apply(lambda x: 'Yes' if x == 1 else 'No', 'Serious'))

    data.show(5)
```

| Date | Time | Area | Serious | Type of crime | Vict Age | Vict Sex | Weapon |
|---|---|---|---|---|---|---|---|
| 5/10/2020 0:00 | 2200 | Central | No | INTIMATE PARTNER - SIMPLE ASSAULT | 30 | F | STRONG-ARM (HANDS, FIST, FEET OR BODILY FORCE) |
| 12/2/2020 0:00 | 2245 | 77th Street | Yes | ROBBERY | 21 | M | VERBAL THREAT |
| 5/1/2020 0:00 | 2330 | 77th Street | Yes | ASSAULT WITH DEADLY WEAPON, AGGRAVATED ASSAULT | 28 | F | KNIFE WITH BLADE OVER 6 INCHES IN LENGTH |
| 1/18/2020 0:00 | 940 | Central | No | BATTERY - SIMPLE ASSAULT | 42 | M | STRONG-ARM (HANDS, FIST, FEET OR BODILY FORCE) |
| 8/27/2020 0:00 | 1255 | Van Nuys | No | BATTERY - SIMPLE ASSAULT | 57 | F | STRONG-ARM (HANDS, FIST, FEET OR BODILY FORCE) |

... (9872 rows omitted)

The same previous method was applied to the column (serious), which displayed the results as two values, 1 and 2. Then in the following code, the value 1 was replaced with the word “yes” and otherwise (the value 2) was replaced with the word “no” and the results were displayed.

```
[ ] # Display all distinct values present in 'Type of crime' column
    unique_values('Type of crime')

array(['ARSON', 'ASSAULT WITH DEADLY WEAPON ON POLICE OFFICER',
       'ASSAULT WITH DEADLY WEAPON, AGGRAVATED ASSAULT',
       'ATTEMPTED ROBBERY', 'BATTERY - SIMPLE ASSAULT',
       'BATTERY ON A FIREFIGHTER', 'BATTERY POLICE (SIMPLE)',
       'BATTERY WITH SEXUAL CONTACT', 'BIKE - STOLEN', 'BOMB SCARE',
       'BRANDISH WEAPON', 'BUNCO, ATTEMPT', 'BUNCO, GRAND THEFT',
       'BUNCO, PETTY THEFT', 'BURGLARY', 'BURGLARY FROM VEHICLE',
       'BURGLARY FROM VEHICLE, ATTEMPTED', 'BURGLARY, ATTEMPTED',
       'CHILD ABUSE (PHYSICAL) - AGGRAVATED ASSAULT',
       'CHILD ABUSE (PHYSICAL) - SIMPLE ASSAULT',
       'CHILD ANNOYING (17YRS & UNDER)', 'CHILD NEGLECT (SEE 300 W.I.C.)',
       'CHILD PORNOGRAPHY', 'CHILD STEALING', 'CONTEMPT OF COURT',
       'CREDIT CARDS, FRAUD USE ($950 & UNDER', 'CRIMINAL HOMICIDE',
       'CRIMINAL THREATS - NO WEAPON DISPLAYED',
       'CRM AGNST CHLD (13 OR UNDER) (14-15 & SUSP 10 YRS OLDER)',
       'CRUELTY TO ANIMALS',
       'DEFRAUDING INNKEEPER/THEFT OF SERVICES, $950 & UNDER',
       'DISCHARGE FIREARMS/SHOTS FIRED', 'DISTURBING THE PEACE',
       'DOCUMENT FORGERY / STOLEN FELONY',
       'EMBEZZLEMENT, GRAND THEFT ($950.01 & OVER)', 'EXTORTION',
       'FALSE IMPRISONMENT', 'HUMAN TRAFFICKING - COMMERCIAL SEX ACTS',
       'HUMAN TRAFFICKING - INVOLUNTARY SERVITUDE', 'INDECENT EXPOSURE',
       'INTIMATE PARTNER - AGGRAVATED ASSAULT',
       'INTIMATE PARTNER - SIMPLE ASSAULT', 'KIDNAPPING',
       'KIDNAPPING - GRAND ATTEMPT',
       'LETTERS, LEWD  -  TELEPHONE CALLS, LEWD', 'LEWD CONDUCT',
       'LEWD/LASCIVIOUS ACTS WITH CHILD', 'MANSLAUGHTER, NEGLIGENT',
       'ORAL COPULATION', 'OTHER ASSAULT', 'OTHER MISCELLANEOUS CRIME',
       'PEEPING TOM', 'PURSE SNATCHING', 'RAPE, ATTEMPTED',
       'RAPE, FORCIBLE', 'RESISTING ARREST', 'ROBBERY',
```

```
[ ] # Combining variations into more common forms
    type_mapping = {'ASSAULT WITH DEADLY WEAPON ON POLICE OFFICER': 'VIOLENCE', 'ASSAULT WITH DEADLY WEAPON, AGGRAVATED ASSAULT': 'VIOLENCE',
          'BATTERY - SIMPLE ASSAULT': 'VIOLENCE', 'BATTERY WITH SEXUAL CONTACT': 'BEHAVIORAL CRIME','BATTERY POLICE (SIMPLE)': 'BEHAVIORAL CRIME',
          'BATTERY ON A FIREFIGHTER': 'VIOLENCE', 'CHILD ABUSE (PHYSICAL) - AGGRAVATED ASSAULT': 'VIOLENCE', 'CHILD ABUSE (PHYSICAL) - SIMPLE ASSAULT': 'BEHAVIORAL CRIME',
          'CRIMINAL HOMICIDE': 'VIOLENCE', 'CRIMINAL THREATS - NO WEAPON DISPLAYED': 'VIOLENCE', 'KIDNAPPING': 'VIOLENCE',  'RAPE, ATTEMPTED': 'TERRORIZATION',
          'RAPE, FORCIBLE' :  'VIOLENCE', 'ROBBERY':'VIOLENCE', 'SEXUAL PENETRATION W/FOREIGN OBJECT':'VIOLENCE', 'ATTEMPTED ROBBERY': 'THEFT', 'BURGLARY': 'THEFT',
          'BURGLARY FROM VEHICLE': 'THEFT', 'BURGLARY FROM VEHICLE, ATTEMPTED': 'THEFT', 'SHOPLIFTING - PETTY THEFT ($950 & UNDER)': 'THEFT',
          'SHOPLIFTING-GRAND THEFT ($950.01 & OVER)': 'THEFT', 'THEFT FROM MOTOR VEHICLE - GRAND ($950.01 AND OVER)': 'THEFT','HUMAN TRAFFICKING - INVOLUNTARY SERVITUDE':'VIOLENCE',
          'THEFT FROM MOTOR VEHICLE - PETTY ($950 & UNDER)': 'THEFT', 'THEFT FROM PERSON - ATTEMPT': 'THEFT', 'THEFT OF IDENTITY': 'THEFT',
          'THEFT PLAIN - PETTY ($950 & UNDER)': 'THEFT', 'THEFT, PERSON': 'THEFT','THEFT-GRAND ($950.01 & OVER)EXCPT,GUNS,FOWL,LIVESTK,PROD': 'THEFT',
          'ARSON': 'PROPERTY DAMAGES', 'VANDALISM - FELONY ($400 & OVER, ALL CHURCH VANDALISMS)': 'PROPERTY DAMAGES','VANDALISM - MISDEAMEANOR ($399 OR UNDER)': 'PROPERTY DAMAGES',
          'BIKE - STOLEN' :'THEFT','BOMB SCARE': 'TERRORIZATION', 'BRANDISH WEAPON': 'TERRORIZATION', 'BUNCO, ATTEMPT':'THEFT', 'BUNCO, GRAND THEFT':'THEFT', 'BUNCO, PETTY THEFT':'THEFT',
          'BURGLARY, ATTEMPTED' :'THEFT', 'CHILD ANNOYING (17YRS & UNDER)': 'BEHAVIORAL CRIME','CHILD NEGLECT (SEE 300 W.I.C.)': 'BEHAVIORAL CRIME',
          'CHILD PORNOGRAPHY' : 'BEHAVIORAL CRIME', 'CHILD STEALING':'TERRORIZATION', 'CONTEMPT OF COURT': 'BEHAVIORAL CRIME', 'COUNTERFEIT': 'THEFT',
          'CRM AGNST CHLD (13 OR UNDER) (14-15 & SUSP 10 YRS OLDER)': 'BEHAVIORAL CRIME', 'DISCHARGE FIREARMS/SHOTS FIRED': 'VIOLENCE', 'DISHONEST EMPLOYEE - GRAND THEFT': 'THEFT',
          'DISTURBING THE PEACE': 'TERRORIZATION', 'DOCUMENT FORGERY / STOLEN FELONY': 'THEFT','EMBEZZLEMENT, GRAND THEFT ($950.01 & OVER)': 'THEFT', 'EXTORTION':'BEHAVIORAL CRIME',
          'FAILURE TO YIELD': 'BEHAVIORAL CRIME', 'FALSE IMPRISONMENT':'VIOLENCE','HUMAN TRAFFICKING - COMMERCIAL SEX ACTS':'TERRORIZATION','INTIMATE PARTNER - AGGRAVATED ASSAULT': 'VIOLENCE',
          'INTIMATE PARTNER - SIMPLE ASSAULT': 'TERRORIZATION','LETTERS, LEWD  -  TELEPHONE CALLS, LEWD':'BEHAVIORAL CRIME', 'LEWD CONDUCT': 'BEHAVIORAL CRIME',
          'LEWD/LASCIVIOUS ACTS WITH CHILD': 'VIOLENCE','ORAL COPULATION': 'BEHAVIORAL CRIME', 'CONTRIBUTING': 'THEFT','INDECENT EXPOSURE':'BEHAVIORAL CRIME',
          'OTHER ASSAULT': 'VIOLENCE', 'OTHER MISCELLANEOUS CRIME':'TERRORIZATION','PANDERING':'BEHAVIORAL CRIME','PEEPING TOM': 'BEHAVIORAL CRIME','PICKPOCKET': 'THEFT',
          'PIMPING': 'BEHAVIORAL CRIME', 'RECKLESS DRIVING': 'PROPERTY DAMAGES','RESISTING ARREST': 'BEHAVIORAL CRIME', 'SEX,UNLAWFUL(INC MUTUAL CONSENT, PENETRATION W/ FRGN OBJ':'BEHAVIORAL CRIME',
          'SHOTS FIRED AT INHABITED DWELLING':'VIOLENCE','SHOTS FIRED AT MOVING VEHICLE, TRAIN OR AIRCRAFT':'VIOLENCE','SODOMY/SEXUAL CONTACT B/W PENIS OF ONE PERS TO ANUS OTH':'BEHAVIORAL CRIME',
          'STALKING':'TERRORIZATION','THEFT FROM MOTOR VEHICLE - ATTEMPT': 'THEFT','THREATENING PHONE CALLS/LETTERS':'TERRORIZATION','THROWING OBJECT AT MOVING VEHICLE': 'VIOLENCE',
          'TRESPASSING': 'VIOLENCE','UNAUTHORIZED COMPUTER ACCESS':'THEFT','VEHICLE - ATTEMPT STOLEN': 'THEFT','VEHICLE - STOLEN': 'THEFT','DEFRAUDING INNKEEPER/THEFT OF SERVICES, $950 & UNDER':'THEFT',
          'VEHICLE, STOLEN - OTHER (MOTORIZED SCOOTERS, BIKES, ETC)': 'THEFT', 'CREDIT CARDS, FRAUD USE ($950 & UNDER':'THEFT', 'CRUELTY TO ANIMALS':'VIOLENCE',
          'VIOLATION OF COURT ORDER':'BEHAVIORAL CRIME','VIOLATION OF RESTRAINING ORDER':'BEHAVIORAL CRIME','VIOLATION OF TEMPORARY RESTRAINING ORDER':'BEHAVIORAL CRIME',
          'KIDNAPPING - GRAND ATTEMPT': 'VIOLENCE', 'MANSLAUGHTER, NEGLIGENT':'VIOLENCE','PURSE SNATCHING': 'THEFT', 'TELEPHONE PROPERTY - DAMAGE': 'PROPERTY DAMAGES', 'THEFT PLAIN - ATTEMPT':'THEFT'
    }
    data = data.with_columns(
        'Type of crime', rename_values(data.column('Type of crime'), type_mapping))
    unique_values('Type of crime')

array(['BEHAVIORAL CRIME', 'PROPERTY DAMAGES', 'TERRORIZATION', 'THEFT',
       'VIOLENCE'],
      dtype='<U16')
```

Here too, the same previous method was applied to the column (type of crime), which showed that the results were very, very large. Therefore, we created the following code and classified crimes into selected categories and types, and we now have only five classifications

```
[ ] # Display all distinct values present in 'Weapon' column
    unique_values('Weapon')
```

```
array(['AIR PISTOL/REVOLVER/RIFLE/BB GUN', 'AXE',
       'BELT FLAILING INSTRUMENT/CHAIN', 'BLUNT INSTRUMENT', 'BOARD',
       'BOMB THREAT', 'BOTTLE', 'BOWIE KNIFE', 'BRASS KNUCKLES',
       'CAUSTIC CHEMICAL/POISON', 'CLEAVER', 'CLUB/BAT',
       'CONCRETE BLOCK/BRICK', 'DEMAND NOTE', 'DIRK/DAGGER',
       'DOG/ANIMAL (SIC ANIMAL ON)', 'EXPLOXIVE DEVICE', 'FIRE',
       'FIXED OBJECT', 'FOLDING KNIFE', 'GLASS', 'HAMMER', 'HAND GUN',
       'HECKLER & KOCH 93 SEMIAUTOMATIC ASSAULT RIFLE', 'ICE PICK',
       'KITCHEN KNIFE', 'KNIFE WITH BLADE 6INCHES OR LESS',
       'KNIFE WITH BLADE OVER 6 INCHES IN LENGTH', 'LIQUOR/DRUGS',
       'MACE/PEPPER SPRAY', 'MACHETE', 'MARTIAL ARTS WEAPONS',
       'OTHER CUTTING INSTRUMENT', 'OTHER FIREARM', 'OTHER KNIFE',
       'PHYSICAL PRESENCE', 'PIPE/METAL PIPE', 'RAZOR', 'RAZOR BLADE',
       'REVOLVER', 'RIFLE', 'ROCK/THROWN OBJECT', 'ROPE/LIGATURE',
       'SCALDING LIQUID', 'SCISSORS', 'SCREWDRIVER',
       'SEMI-AUTOMATIC PISTOL', 'SEMI-AUTOMATIC RIFLE', 'SHOTGUN',
       'SIMULATED GUN', 'STICK',
       'STRONG-ARM (HANDS, FIST, FEET OR BODILY FORCE)', 'STUN GUN',
       'SWITCH BLADE', 'SWORD', 'TIRE IRON', 'TOY GUN', 'UNKNOWN FIREARM',
       'UNKNOWN TYPE CUTTING INSTRUMENT', 'VEHICLE', 'VERBAL THREAT'],
      dtype='<U46')
```

.

```
# Combining variations into more common forms
weapon_mapping = {
    'AIR PISTOL/REVOLVER/RIFLE/BB GUN': 'FIREARM','SCISSORS' : 'SHARP TOOL','GLASS' : 'SHARP TOOL',
    'HAND GUN': 'FIREARM', 'AXE':'SHARP TOOL','RAZOR BLADE': 'SHARP TOOL',
    'SHOTGUN': 'FIREARM', 'ICE PICK':'SHARP TOOL','CLEAVER' : 'SHARP TOOL',
    'PIPE/METAL PIPE': 'SHARP TOOL',
    'FOLDING KNIFE': 'SHARP TOOL','HECKLER & KOCH 93 SEMIAUTOMATIC ASSAULT RIFLE': 'FIREARM',
    'KITCHEN KNIFE': 'SHARP TOOL',
    'KNIFE' : 'SHARP TOOL', 'SCREWDRIVER': 'SHARP TOOL',
    'KNIFE WITH BLADE 6INCHES OR LESS': 'SHARP TOOL', 'DEMAND NOTE':'THREAT',
    'KNIFE WITH BLADE OVER 6 INCHES IN LENGTH': 'SHARP TOOL',
    'MACHETE': 'SHARP TOOL', 'BRASS KNUCKLES': 'HEAVY OBJECT',
    'OTHER CUTTING INSTRUMENT': 'SHARP TOOL',
    'OTHER FIREARM': 'FIREARM',
    'OTHER KNIFE': 'SHARP TOOL', 'STUN GUN': 'FIREARM',
    'REVOLVER': 'FIREARM','SEMI-AUTOMATIC RIFLE':'FIREARM',
    'SEMI-AUTOMATIC PISTOL': 'FIREARM',
    'SIMULATED GUN': 'FIREARM','DIRK/DAGGER':'SHARP TOOL',
    'SWITCH BLADE': 'SHARP TOOL', 'TIRE IRON': 'HEAVY OBJECT',
    'SWORD': 'SHARP TOOL', 'BOWIE KNIFE':'SHARP TOOL',
    'UNKNOWN FIREARM': 'FIREARM','MARTIAL ARTS WEAPONS':'PHYSICAL VIOLENCE',
    'UNKNOWN TYPE CUTTING INSTRUMENT': 'SHARP TOOL',
    'RAZOR': 'SHARP TOOL','DOG/ANIMAL (SIC ANIMAL ON)':'PHYSICAL VIOLENCE',
    'BELT FLAILING INSTRUMENT/CHAIN':'ROPE/LIGATURE', 'BLUNT INSTRUMENT': 'SHARP TOOL',
    'SCALDING LIQUID': 'BORN', 'EXPLOXIVE DEVICE':'FIREARM',
    'FIRE': 'BORN', 'CLUB/BAT': 'HEAVY OBJECT', 'TOY GUN':'FIREARM',
    'CAUSTIC CHEMICAL/POISON':'BORN', 'RIFLE':'FIREARM',
    'STRONG-ARM (HANDS, FIST, FEET OR BODILY FORCE)' :'PHYSICAL VIOLENCE',
    'PHYSICAL PRESENCE': 'PHYSICAL VIOLENCE',
    'BOARD': 'HEAVY OBJECT', 'HAMMER': 'HEAVY OBJECT',
    'BOTTLE': 'HEAVY OBJECT', 'FIXED OBJECT': 'HEAVY OBJECT',
    'CONCRETE BLOCK/BRICK': 'HEAVY OBJECT', 'LIQUOR/DRUGS':'BORN',
    'ROCK/THROWN OBJECT': 'HEAVY OBJECT', 'MACE/PEPPER SPRAY':'BORN',
    'BOMB THREAT': 'THREAT', 'STICK':'HEAVY OBJECT', 'VEHICLE': 'HEAVY OBJECT',
    'VERBAL THREAT': 'THREAT',

}
data = data.with_columns("Weapon", rename_values(data.column("Weapon"), weapon_mapping))
unique_values('Weapon')
```

```
array(['BORN', 'FIREARM', 'HEAVY OBJECT', 'PHYSICAL VIOLENCE',
       'ROPE/LIGATURE', 'SHARP TOOL', 'THREAT'],
      dtype='<U17')
```

This time, we applied the method to the (weapon) column, and it showed us many, many results, so we classified them and reduced them to seven categories, as we did previously with the types of crimes.

```
[ ] # Dealting unwanted values
    data = data.where('Vict Sex', lambda y: y not in ["X", np.nan])
    # Replacing values for better readability
    data = data.with_column('Vict Sex', data.apply(lambda Z: 'MALE' if Z == 'M' else 'FEMALE', 'Vict Sex'))

    data.show(5)
```

| Date | Time | Area | Serious | Type of crime | Vict Age | Vict Sex | Weapon |
|---|---|---|---|---|---|---|---|
| 5/10/2020 0:00 | 2200 | Central | No | TERRORIZATION | 30 | FEMALE | PHYSICAL VIOLENCE |
| 12/2/2020 0:00 | 2245 | 77th Street | Yes | VIOLENCE | 21 | MALE | THREAT |
| 5/1/2020 0:00 | 2330 | 77th Street | Yes | VIOLENCE | 28 | FEMALE | SHARP TOOL |
| 1/18/2020 0:00 | 940 | Central | No | VIOLENCE | 42 | MALE | PHYSICAL VIOLENCE |
| 8/27/2020 0:00 | 1255 | Van Nuys | No | VIOLENCE | 57 | FEMALE | PHYSICAL VIOLENCE |

... (9816 rows omitted)

By moving to the (Vict sex) column, the first line removes any unwanted values such as X or Nan. The next line unifies the gender form into the word "Male" or the word "Female".

```
[ ] # Reformat Time from likely integer format into more standard '24-hour time' format
    def format_time(value):
        value = str(value).zfill(4)
        hour = int(value[:2])
        minute = int(value[2:])
        formatted_time = f"{hour:02}:{minute:02}"
        return formatted_time

    data = data.with_column("Time 24-form", data.apply(format_time, "Time"))
    data.show(5)
```

| Date | Time | Area | Serious | Type of crime | Vict Age | Vict Sex | Weapon | Time 24-form |
|---|---|---|---|---|---|---|---|---|
| 5/10/2020 0:00 | 2200 | Central | No | TERRORIZATION | 30 | FEMALE | PHYSICAL VIOLENCE | 22:00 |
| 12/2/2020 0:00 | 2245 | 77th Street | Yes | VIOLENCE | 21 | MALE | THREAT | 22:45 |
| 5/1/2020 0:00 | 2330 | 77th Street | Yes | VIOLENCE | 28 | FEMALE | SHARP TOOL | 23:30 |
| 1/18/2020 0:00 | 940 | Central | No | VIOLENCE | 42 | MALE | PHYSICAL VIOLENCE | 09:40 |
| 8/27/2020 0:00 | 1255 | Van Nuys | No | VIOLENCE | 57 | FEMALE | PHYSICAL VIOLENCE | 12:55 |

... (9816 rows omitted)

This code uses a column (time) and changes its format from integer format to time format in 24-hour format, then creates a new column in the table and places the new values in it.

```
[ ] # Takes 'Date' column that has both date and time and creates a new column with only the date
    def extract_date_only(date_str):
        return datetime.strptime(date_str, "%m/%d/%Y %H:%M").strftime("%m/%d/%Y")

    data = data.with_column("Date", data.apply(lambda d: extract_date_only(d), "Date"))
    data.show(5)
```

| Date | Time | Area | Serious | Type of crime | Vict Age | Vict Sex | Weapon | Time 24-form |
|---|---|---|---|---|---|---|---|---|
| 05/10/2020 | 2200 | Central | No | TERRORIZATION | 30 | FEMALE | PHYSICAL VIOLENCE | 22:00 |
| 12/02/2020 | 2245 | 77th Street | Yes | VIOLENCE | 21 | MALE | THREAT | 22:45 |
| 05/01/2020 | 2330 | 77th Street | Yes | VIOLENCE | 28 | FEMALE | SHARP TOOL | 23:30 |
| 01/18/2020 | 940 | Central | No | VIOLENCE | 42 | MALE | PHYSICAL VIOLENCE | 09:40 |
| 08/27/2020 | 1255 | Van Nuys | No | VIOLENCE | 57 | FEMALE | PHYSICAL VIOLENCE | 12:55 |

... (9816 rows omitted)

This section uses the (Date) column, which contains the time and date, and changes its content to include the date only.

# 5.DATA ANALYSIS

```
# The frequency of different 'Type of crime'
num_of_typs = data.group('Type of crime')
num_of_typs
```

| Type of crime | count |
|---|---|
| BEHAVIORAL CRIME | 713 |
| PROPERTY DAMAGES | 516 |
| TERRORIZATION | 1844 |
| THEFT | 1454 |
| VIOLENCE | 5294 |

```
# The frequency of different 'Area'
num_fo_crm_in_area = data.group('Area')
num_fo_crm_in_area
```

| Area | count |
|---|---|
| 77th Street | 309 |
| Central | 402 |
| Devonshire | 188 |
| Foothill | 189 |
| Harbor | 3822 |
| Hollenbeck | 100 |
| Hollywood | 412 |

```
# The frequency of 'Serious' column
is_it_serious = data.group('Serious')
is_it_serious
```

| Serious | count |
|---|---|
| No | 5492 |
| Yes | 4329 |

```
# The frequency of 'Vict Sex' column
vict_sex = data.group('Vict Sex')
vict_sex
```

| Vict Sex | count |
|---|---|
| FEMALE | 5135 |
| MALE | 4686 |

```
# The frequency of 'Weapon' column
Woeben_data= data.group('Weapon')
Woeben_data.show()
```

| Weapon | count |
|---|---|
| BORN | 94 |
| FIREARM | 724 |
| HEAVY OBJECT | 329 |
| PHYSICAL VIOLENCE | 7517 |
| ROPE/LIGATURE | 18 |
| SHARP TOOL | 644 |
| THREAT | 495 |

The group() function is used to categorize and count the occurrences of different values within a specific column. In this case, it helps analyze crime data by grouping incidents based on various factors such as crime type, location (area), severity, victim gender, and weapon used.

```python
# Categorize the time of crimes into different time periods
# Defining Time Labels
time_labels = {
    0: '12AM - 3AM',
    300: '3AM - 6AM',
    600: '6AM - 9AM',
    900: '9AM - 12PM',
    1200: '12PM - 3PM',
    1500:'3PM - 6PM',
    1800: '6PM - 9PM',
    2100: '9PM - 12AM'
}
# Assigning Time Bins to Crimes
data_time = data.select("Time")
bins = [0, 300, 600, 900, 1200, 1500, 1800, 2100, 2400]
def assign_time_bin(time):
    for i in range(len(bins) - 1):
        if bins[i] <= time < bins[i + 1]:
            return time_labels.get(bins[i], "Unknown")
    return time_labels.get(bins[-1], "Unknown")
data_time_binned = data_time.with_column("Time period", data_time.apply(assign_time_bin, "Time"))

data_time_binned.group("Time period")
```

| Time period | count |
|---|---|
| 12AM - 3AM | 1034 |
| 12PM - 3PM | 1502 |
| 3AM - 6AM | 478 |

categorizes crime times into specific time periods and calculates the frequency of crimes occurring in each period.

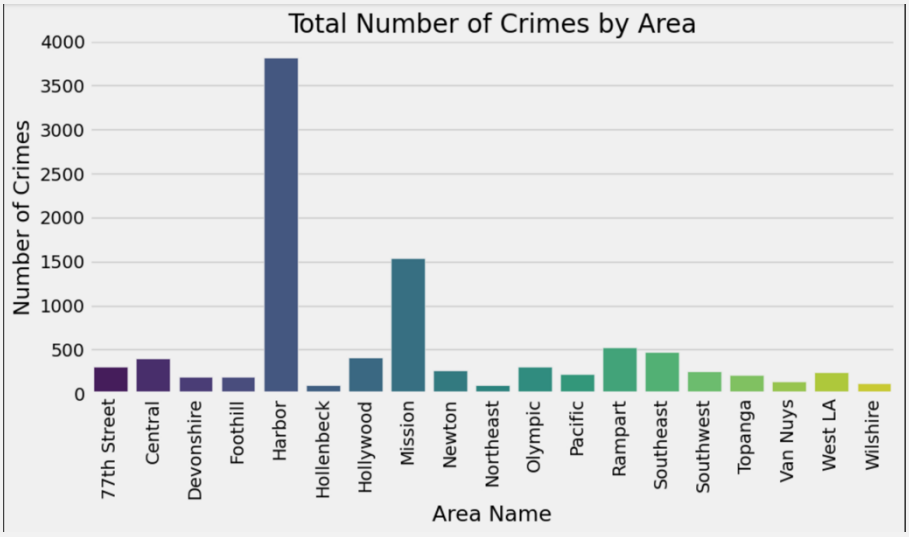
```python
# Categorize victim ages into groups and then count the number of crimes within each age group
age_bins = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
age_labels = ['0-9', '10-19', '20-29', '30-39', '40-49', '50-59', '60-69', '70-79', '80-89', '90-100']
def assign_age_group(age):
  for i in range(len(age_bins) - 1):
    if age_bins[i] <= age < age_bins[i + 1]:
      return age_labels[i]
  return age_labels[-1]
crimes_by_age = data.with_column("Age Group", data.apply(assign_age_group, "Vict Age"))
crimes_by_age = crimes_by_age.group("Age Group")
crimes_by_age.show()
```

| Age Group | count |
|---|---|
| 0-9 | 117 |
| 10-19 | 823 |
| 20-29 | 2392 |
| 30-39 | 2416 |
| 40-49 | 1747 |

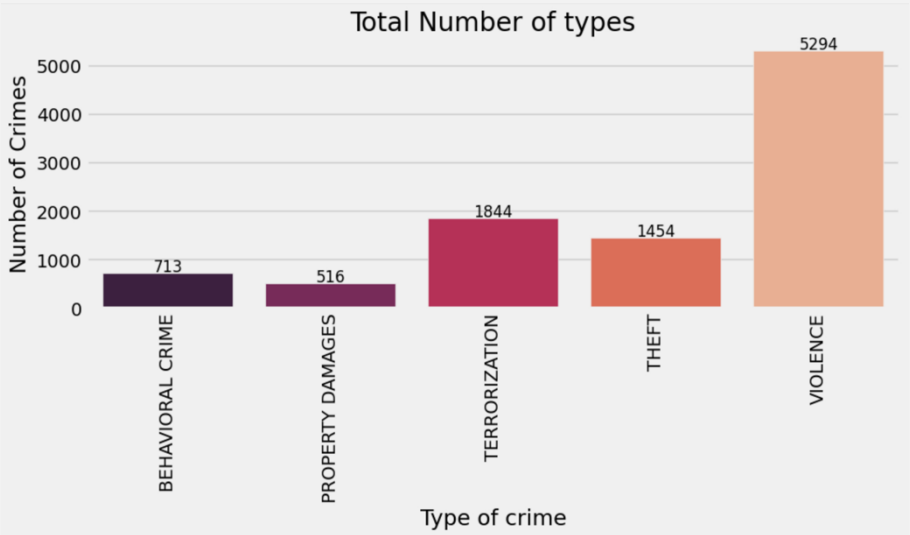
categorizes victim ages into groups and counts the number of crimes for each age range.

# 6.VISUALISATION

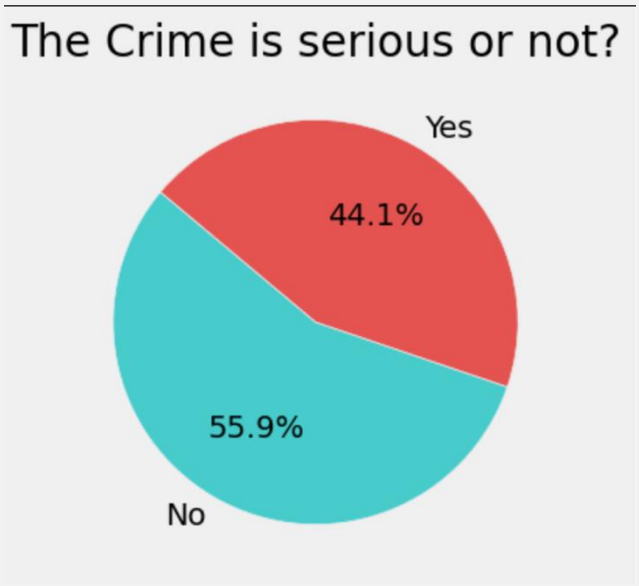
## 6.1. BASED ON ONE VARIABLE



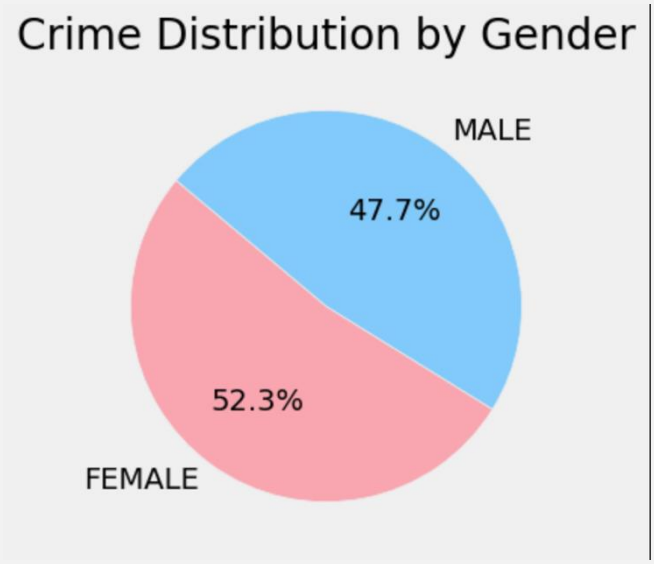

This bar chart shows the total number of crimes in different areas. The x-axis represents different areas, while the y-axis represents the number of crimes. The area Harbor has the highest crime rate, followed by Mission, while other areas have significantly fewer crimes.



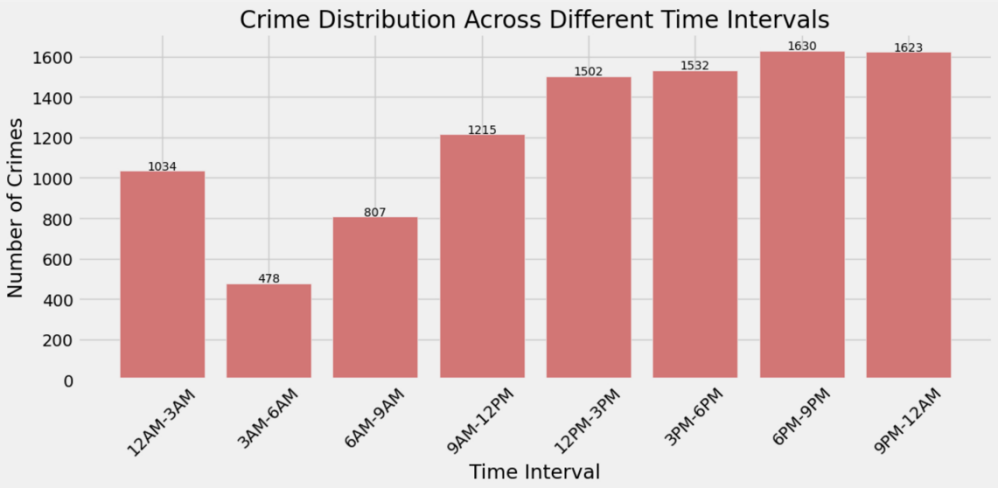

This bar chart displays the total number of crimes by type. The x-axis represents different crime categories, while the y-axis shows the number of occurrences. The highest crime type is violence (5294 cases)



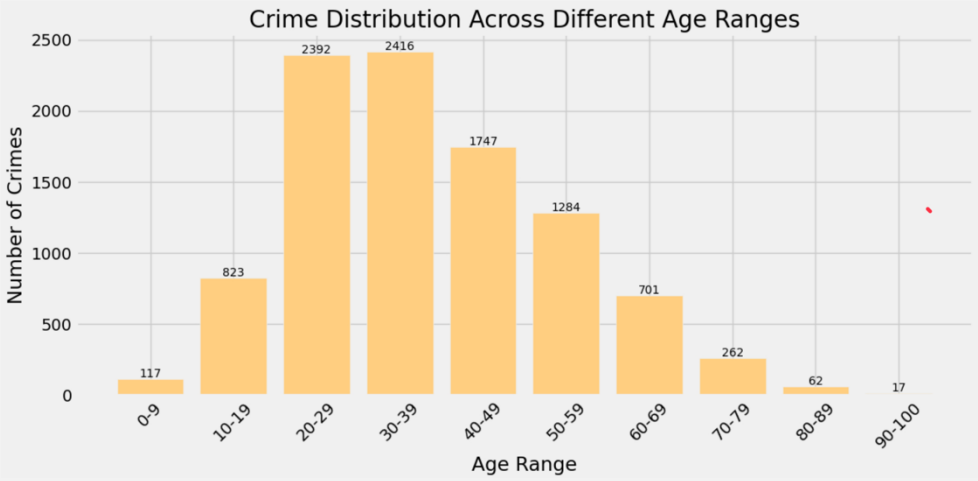

This pie chart represents the proportion of serious vs. non-serious crimes.



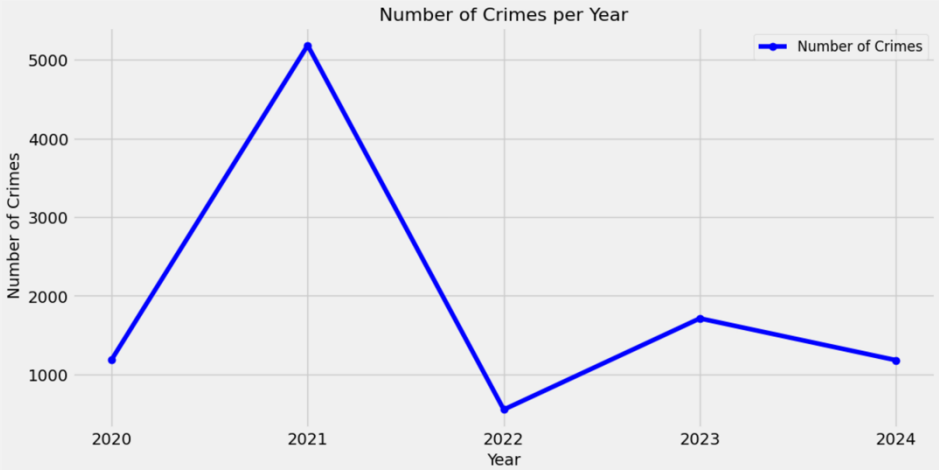

This pie chart shows the distribution of crimes by gender of the victims.




This bar chart displays the distribution of crimes across different time intervals. Crime rates are lowest between 3 AM - 6 AM (478 crimes).




This bar chart shows the distribution of crimes across different age groups. The highest number of crimes occurs in the 20-29 (2392 crimes) and 30-39 (2416 crimes) age groups.

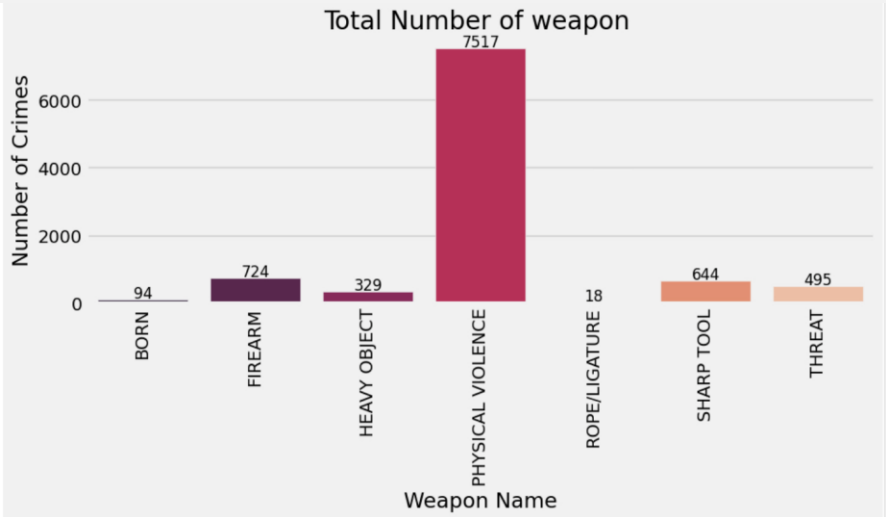



This line chart shows the number of crimes per year from 2020 to 2024

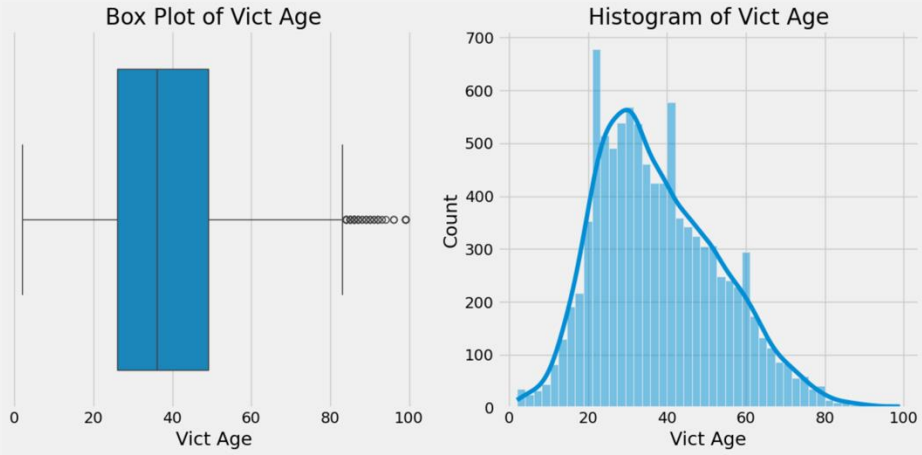
Crime numbers peaked in 2021, reaching over 5000 incidents.

A sharp decline in 2022 shows the lowest crime rate.

This bar chart represents the total number of crimes by weapon type.Physical violence is the most common type, with 7517 incidents.




This bar chart represents the total number of crimes by weapon type.
Physical violence is the most common type, with 7517 incidents.

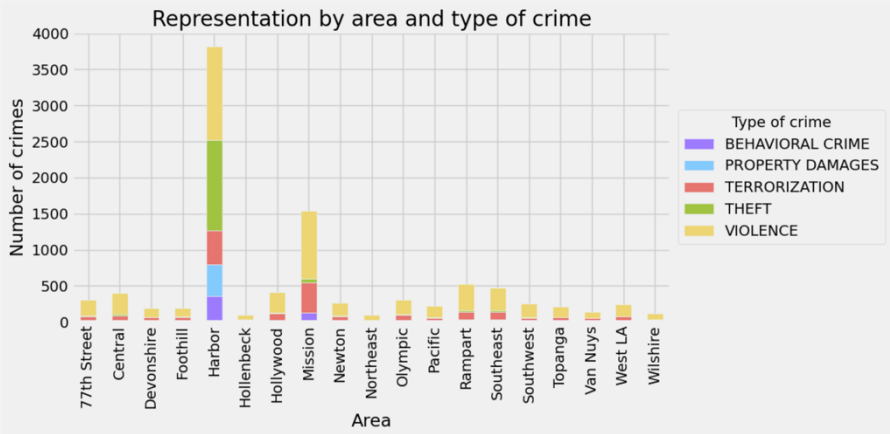



These two plots represent the distribution of victim ages in crime data.

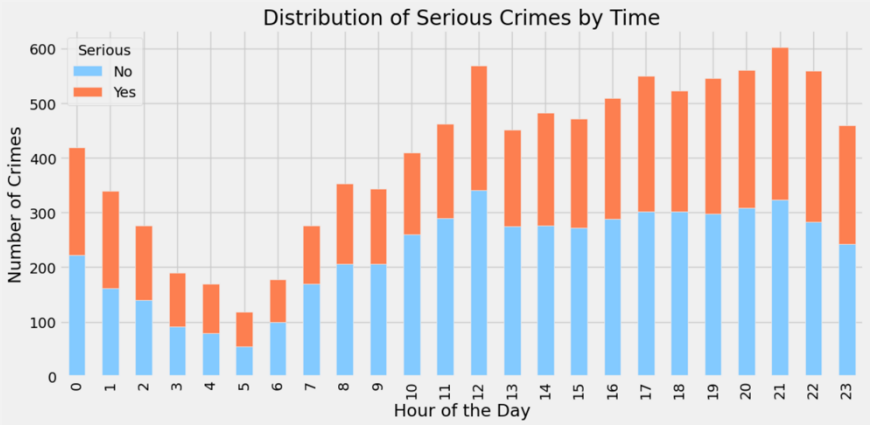
Box Plot : Shows the median, quartiles, and outliers of victim ages. The data is skewed right, with some outliers above age 80.

Histogram : Displays the frequency distribution of victim ages. Most victims are between 10-40 years old, with a peak around 20 years old.

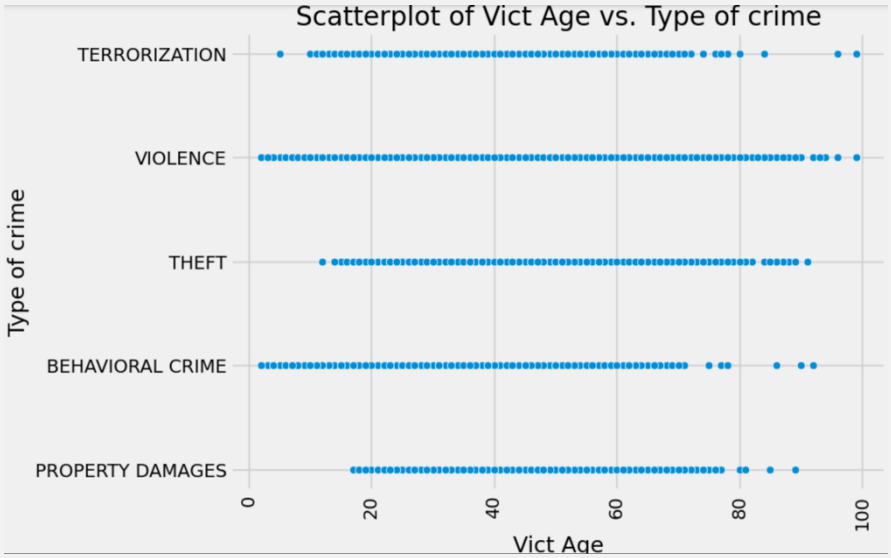
## 6.2. BASED ON MULTI VARIABLE




This stacked bar chart shows the distribution of crimes by area and type.
Harbor and Mission have the highest crime rates, with theft and violence being the most common crimes.
Most other areas have relatively lower crime rates, dominated by violence (yellow) and terrorization (red).
The least common crimes across all areas are behavioral crimes (purple) and rope/ligature-related crimes.

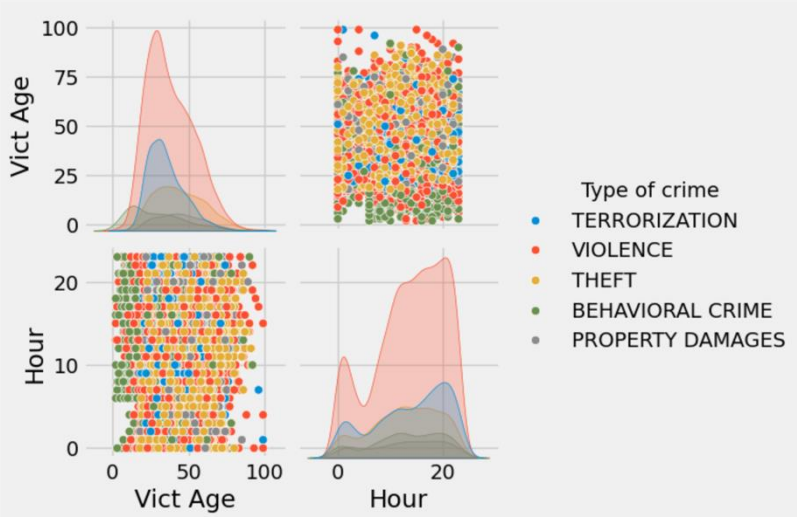



This stacked bar chart shows the distribution of serious and non-serious crimes by hour of the day.
Early morning (12 AM - 6 AM) has fewer crimes, but a higher proportion of serious crimes (orange).
Crimes increase throughout the day, peaking between 6 PM - 11 PM, with both serious (orange) and non-serious (blue) crimes rising.
The highest crime hour is 10 PM - 11 PM, showing a peak in overall incidents.

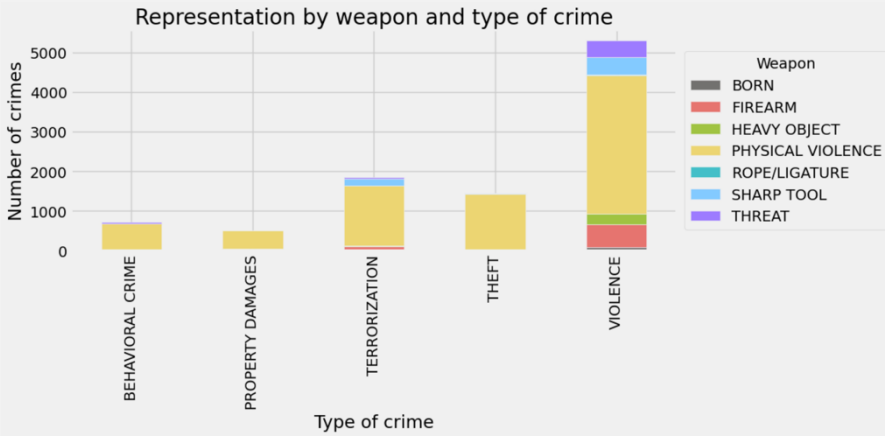



This scatter plot visualizes the relationship between victim age and type of crime.
Crimes are distributed across all age groups, but some crime types have more victims in specific age ranges.
Violence and terrorization crimes occur at all ages, while property damages and behavioural crimes are less frequent in older victims.

The distribution suggests that younger individuals are more affected by theft and behavioural crimes, while violence remains common across all ages.




This pair plot visualizes the relationship between victim age, time of crime (hour), and crime type.
The top-left density plot shows that most victims are younger, with a peak around 20-30 years old.
The bottom-right density plot indicates that crimes peak between 6 PM - 11 PM, especially for violence (red).
The scatterplots show that all crime types occur at various ages and times, but violence and theft are more frequent.




This stacked bar chart visualizes the distribution of weapons used in different types of crimes.
Physical violence (yellow) is the most common weapon type across all crime categories, especially in violence and theft-related crimes.
Firearms (red) are mainly associated with violence and terrorization crimes.
Sharp tools (blue) and threats (purple) also contribute, but at a lower rate.

# 7.MODELLING

```
# Copying clean data and dropping the columns we do not need
modle_data = clean_data.drop(['Time','Status','Time 24-form'], axis=1, errors='ignore')
# Select Hour column from byHour
modle_data['Hour'] = byHour['Hour']
modle_data.head()
```

| | Date | Area | Serious | Type of crime | Vict Age | Vict Sex | Weapon | Hour |
|---|---|---|---|---|---|---|---|---|
| 0 | 05/10/2020 | Central | No | TERRORIZATION | 30 | FEMALE | PHYSICAL VIOLENCE | 22 |
| 1 | 12/02/2020 | 77th Street | Yes | VIOLENCE | 21 | MALE | THREAT | 22 |
| 2 | 05/01/2020 | 77th Street | Yes | VIOLENCE | 28 | FEMALE | SHARP TOOL | 23 |
| 3 | 01/18/2020 | Central | No | VIOLENCE | 42 | MALE | PHYSICAL VIOLENCE | 9 |
| 4 | 08/27/2020 | Van Nuys | No | VIOLENCE | 57 | FEMALE | PHYSICAL VIOLENCE | 12 |

This code cleans and preprocesses the dataset:

1.Drops unnecessary columns ('Time', 'Status', 'Time 24-form') from clean_data.

2.Adds an ‘Hour’ column from byHour['Hour'] to the new dataset modle_data.

```
# Feature Selection and Preprocessing:
features = ['Area', 'Date', 'Vict Sex', 'Hour', 'Serious', 'Weapon']
targets = ['Type of crime', 'Vict Age']

X = modle_data[features]
y = modle_data[targets]
```

Prepare the data for training a machine learning model by defining independent variables (X) as inputs and target variables (y) as outputs for predicting crime type and victim age.

```python
# Data transformtion
# Create LabelEncoder objects for each categorical feature
# Encoding Categorical Features:
categorical_features = ['Area', 'Vict Sex', 'Serious', 'Weapon']
encoders = {}
for feature in categorical_features:
    encoders[feature] = LabelEncoder()
    X[feature] = encoders[feature].fit_transform(X[feature])

# Convert Date to Numerical Features: Year ,Month ,Day
X['Year'] = pd.to_datetime(X['Date'], format='%m/%d/%Y').dt.year
X['Month'] = pd.to_datetime(X['Date'], format='%m/%d/%Y').dt.month
X['Day'] = pd.to_datetime(X['Date'], format='%m/%d/%Y').dt.day

X = X.drop('Date', axis=1)


# Feature Scaling:
scaler = MinMaxScaler()
numerical_features = ['Year', 'Month', 'Day', 'Hour']
X[numerical_features] = scaler.fit_transform(X[numerical_features])

# why we used Normalization scaler"MinMaxScaler"?
# -Less sensitive to outliers compared to StandardScaler.
# -Preserves the shape of the original distribution
#   without distorting the relationships between data points
#   This is important for maintaining the underlying patterns in our data.
# -Improved the performance and convergence of our Random Forest model in predicting crime types and victim ages

X.head()
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_gui
  X[feature] = encoders[feature].fit_transform(X[feature])
<ipython-input-49-976dd03423af>:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-ve
  X[feature] = encoders[feature].fit_transform(X[feature])
<ipython-input-49-976dd03423af>:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-ve
  X[feature] = encoders[feature].fit_transform(X[feature])
<ipython-input-49-976dd03423af>:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-ve
  X[feature] = encoders[feature].fit_transform(X[feature])
<ipython-input-49-976dd03423af>:11: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-ve
  X['Year'] = pd.to_datetime(X['Date'], format='%m/%d/%Y').dt.year
```

| | Area | Vict Sex | Hour | Serious | Weapon | Year | Month | Day |
|---|---|---|---|---|---|---|---|---|
| **0** | 1 | 0 | 0.956522 | 0 | 3 | 0.0 | 0.363636 | 0.300000 |
| **1** | 0 | 1 | 0.956522 | 1 | 6 | 0.0 | 1.000000 | 0.033333 |

preprocess the dataset by encoding categorical features, converting dates into numerical values, and normalizing numerical features. This transformation ensures the data is in a suitable format for machine learning models, improving model performance and interpretation.

```python
# Data transformtion
# Create LabelEncoder objects for categorical and numerical target variable

le_crime = LabelEncoder()
y['Type of crime'] = le_crime.fit_transform(y['Type of crime'])

y.head()
```

```
<ipython-input-50-0b67a887f03c>:5: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-ve
  y['Type of crime'] = le_crime.fit_transform(y['Type of crime'])
```

| | Type of crime | Vict Age |
|---|---|---|
| **0** | 2 | 30 |
| **1** | 4 | 21 |
| **2** | 4 | 28 |
| **3** | 4 | 42 |
| **4** | 4 | 57 |

convert categorical values in the “Type of crime” column into numerical values using LabelEncoder, making the data ready for use in machine learning models

that require numerical inputs. The encoded values appear in the table, where crime names have been replaced with numbers.

```
# 5. Split Data into Training and Testing Sets:
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 6. Model Training:
base_model = RandomForestClassifier(n_estimators=100, random_state=42)
model = MultiOutputClassifier(base_model)
model.fit(X_train, y_train)
```

```
MultiOutputClassifier
estimator:
RandomForestClassifier
RandomForestClassifier
```

**split the dataset into training and testing sets** and then **train a multi-output classification model** using a **Random Forest Classifier**. The train_test_split function divides the data (80% for training, 20% for testing), and the MultiOutputClassifier is used to handle multiple target variables simultaneously.

### Why Random Forest Classifier model?

1. Handles Both Categorical and Numerical Features: The dataset in this case contains both categorical features (like area, gender, weapon type) and numerical features (like victim age, hour). Random Forest can effectively handle both types of features without requiring extensive preprocessing.
2. Multi-Output Prediction: The MultiOutputClassifier wrapper allows Random Forest to predict multiple target variables simultaneously, which is essential for this task.
3. Classification Task: The task of predicting the type of crime is a classification problem, and Random Forest is a strong classification algorithm
4. Regression Task: Predicting the victim's age is a regression problem, and while Random Forest is primarily used for classification, it can also be used for regression tasks.

```python
# 05/10/2020  Central No  TERRORIZATION 30  FEMALE  PHYSICAL VIOLENCE 22
# 8. Prediction on New Data:
new_data = pd.DataFrame({'Area': ['Central'], 'Year': [2020], 'Month': [10], 'Day': [5], 'Hour': [22],
                         'Vict Sex': ['FEMALE'], 'Serious': ['No'], 'Weapon': ['PHYSICAL VIOLENCE']})

# Reorder the columns to match X_train
new_data = new_data[['Area', 'Vict Sex', 'Hour', 'Serious', 'Weapon', 'Year', 'Month', 'Day']]

# Transform new data using the encoders fitted during training

for feature in categorical_features:
    # Handle unknown labels by adding them to the encoder's classes
    if new_data[feature][0] not in encoders[feature].classes_:
        encoders[feature].classes_ = np.append(encoders[feature].classes_, new_data[feature][0])
    new_data[feature] = encoders[feature].transform(new_data[feature])

new_data[numerical_features] = scaler.transform(new_data[numerical_features])

predicted_values = model.predict(new_data)

# Decode the predictions
predicted_crime_type = le_crime.inverse_transform([predicted_values[0][0]])[0]
predicted_victim_age = predicted_values[0][1]

print(f"\nPredicted Crime Type: {predicted_crime_type}")
print(f"Predicted Victim Age: {predicted_victim_age}")
```

```
Predicted Crime Type: TERRORIZATION
Predicted Victim Age: 27
```

This code aims to **predict the type of crime and the victim's age based on new data** using a previously trained model. The new data is transformed to match the format of the training data, then the same encoding and normalization processes are applied. After that, the data is passed to the model for predictions, and finally, the predictions are decoded to display the results in a readable format.

# 8. CONCLUSION

## KEY FINDINGS:

- **THEFT** and **VIOLENCE** are the most common crimes.
- **Harbor** and **Mission** have high crime rates.
- Crime peaks between **6 PM and 9 PM**.
- **Young adults** and **females** are more often victims.
- **Physical violence** and **firearm** are frequently used weapons.
- Crime rates have generally **decreased** over time.

## MODEL INSIGHTS:

- The Random Forest model can predict crime types and victim ages.
- This information can be used to improve resource allocation and crime prevention strategies.

## RECOMMENDATIONS:

- Focus on high-crime areas, times, and vulnerable demographics.
- Use data-driven strategies for crime prevention.

## FUTURE DIRECTIONS:

- Explore real-time crime prediction and predictive policing.
- Develop risk assessment tools and public awareness programs.